

BECS 12243

Project Report

Modern Residential Electricity Bill Calculator

Group number - 22

EC/2023/071

EC/2023/044

EC/2023/078

EC/2023/014

EC/2023/037

1. Introduction

The implementation of complex electricity tariff structures—specifically the transition to multi-tier slabs and **Time-of-Use (TOU)** rates (effective October 2025)—often leaves consumers confused about their monthly costs. This project provides a robust tool to calculate monthly charges, incorporate solar energy exports (Net Metering), and analyze the financial benefits of renewable energy integration.

2. System Requirements

The system was developed based on the following functional and design requirements:

Functional Requirements

- **Standard Billing:** Calculate energy charges based on tiered consumption slabs (0-30, 31-60, and 60+ units).
- **TOU Billing:** Calculate charges based on specific Day, Peak, and Off-Peak consumption periods.
- **Solar Credit Integration:** Deduct financial credits earned from solar energy exported back to the grid.
- **Bill Generation:** Produce a detailed summary including energy charges, fixed charges, and the final net payable amount.

Non-Functional (OOP) Requirements

- **Encapsulation:** Isolating calculation logic and rate data within specific classes to ensure data integrity.
- **Abstraction:** Providing a simplified interface for complex billing processes.
- **Inheritance:** Using a base Consumer class to allow for future expansion into other categories like 'Religious Institutions'.

3. System Architecture and Design

The application is built using a modular Object-Oriented approach with the following core components:

Class Structure Overview

Class / Interface	Role
BillingService	An interface defining the contract for billing calculations and printing.
TariffRate	A dedicated data class encapsulating all current rates, such as the Rs. 4.50 slab rate or the Rs. 2,100.00 TOU fixed charge.
Consumer	An abstract base class holding shared data like consumerID, name, and solarExportUnits.
StandardConsumer	A subclass implementing specific logic for tiered slab energy charges.
TOUConsumer	A subclass implementing logic for time-based energy charges.
ResidentialBillingEngine	The core engine that aggregates consumer data and tariff rates to compute the final bill.

Code

- ElecBillCalculator.java

```
import java.util.Scanner;

public class ElecBillCalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        TariffRate rates = new TariffRate();
        ResidentialBillingEngine engine = new ResidentialBillingEngine();

        System.out.println("--- Team 22 Electricity Bill Calculator ---");

        System.out.print("Enter Consumer Name: ");
        String name = scanner.nextLine();

        System.out.print("Enter Consumer ID: ");
        String id = scanner.nextLine();

        System.out.print("Enter Solar Export Units (kWh) (0 if none): ");
        double solar = scanner.nextDouble();

        System.out.println("\nSelect Tariff Plan:");
        System.out.println("1. Standard Domestic");
        System.out.println("2. Time-of-Use (TOU)");
        System.out.print("Choice: ");
        int choice = scanner.nextInt();

        Consumer consumer = null;

        if (choice == 1) {
            System.out.print("Enter Total Monthly Consumption (kWh): ");
            double total = scanner.nextDouble();
            consumer = new StandardConsumer(id, name, solar, total);
        } else if (choice == 2) {
            System.out.print("Enter Day Consumption (kWh): ");
            double day = scanner.nextDouble();
            System.out.print("Enter Peak Consumption (kWh): ");
            double peak = scanner.nextDouble();
            System.out.print("Enter Off-Peak Consumption (kWh): ");
            double off = scanner.nextDouble();
            consumer = new TOUConsumer(id, name, solar, day, peak, off);
        } else {
            System.out.println("Invalid choice. Exiting.");
            System.exit(0);
        }

        engine.printBill(consumer, rates);

        scanner.close();
    }
}
```

- **TariffRate.java**

```
public class TariffRate {
    private final double domesticSlab1Rate = 4.50;
    private final double domesticSlab2Rate = 8.00;

    private final double above60Block1Rate = 12.75;
    private final double above60Block2Rate = 18.50;
    private final double above60Block3Rate = 24.00;

    private final double domesticFixedCharge1 = 80.00;
    private final double domesticFixedCharge2 = 210.00;

    private final double TOUDayRate = 35.00;
    private final double TOUPeakRate = 67.00;
    private final double TOUOffPeakRate = 21.00;
    private final double TOUFixedCharge = 2100.00; // [cite: 34]

    private final double solarExportRate = 25.00;

    public double getDomesticSlab1Rate() { return domesticSlab1Rate; }
    public double getDomesticSlab2Rate() { return domesticSlab2Rate; }
    public double getAbove60Block1Rate() { return above60Block1Rate; }
    public double getAbove60Block2Rate() { return above60Block2Rate; }
    public double getAbove60Block3Rate() { return above60Block3Rate; }
    public double getDomesticFixedCharge1() { return domesticFixedCharge1; }
    public double getDomesticFixedCharge2() { return domesticFixedCharge2; }
    public double getTOUDayRate() { return TOUDayRate; }
    public double getTOUPeakRate() { return TOUPeakRate; }
    public double getTOUOffPeakRate() { return TOUOffPeakRate; }
    public double getTOUFixedCharge() { return TOUFixedCharge; }
    public double getSolarExportRate() { return solarExportRate; }
}
```

- **Consumer.java**

```
abstract class Consumer {
    private String consumerID;
    private String name;
    private double solarExportUnits;

    public Consumer(String consumerID, String name, double solarExportUnits){
        this.consumerID = consumerID;
        this.name = name;
        this.solarExportUnits = solarExportUnits;
    }

    public abstract double calculateEnergyCharge(TariffRate rates);

    public abstract double getFixedCharge(TariffRate rates);

    public String getConsumerID() { return consumerID; }
    public String getName() { return name; }
    public double getSolarExportUnits() { return solarExportUnits; }
}
```

- **StandardConsumer.java**

```
class StandardConsumer extends Consumer {
    private double totalConsumption;

    public StandardConsumer(String id, String name, double solarExportUnits,
double totalConsumption) {
        super(id, name, solarExportUnits);
        this.totalConsumption = totalConsumption;
    }

    @Override
    public double calculateEnergyCharge(TariffRate rates) {
        double units = totalConsumption;
        double charge = 0;

        if (units <= 60) {
            if (units <= 30) {
                charge = units * rates.getDomesticSlab1Rate();
            } else {
                charge = (30 * rates.getDomesticSlab1Rate()) +
                    ((units - 30) * rates.getDomesticSlab2Rate());
            }
        } else {
            if (units <= 90) {
                charge = (60 * rates.getAbove60Block1Rate()) +
                    ((units - 60) * rates.getAbove60Block2Rate());
            } else {
                charge = (60 * rates.getAbove60Block1Rate()) +
                    (30 * rates.getAbove60Block2Rate()) +
                    ((units - 90) * rates.getAbove60Block3Rate());
            }
        }
        return charge;
    }

    @Override
    public double getFixedCharge(TariffRate rates) {
        if (totalConsumption <= 30) return rates.getDomesticFixedCharge1();
        return rates.getDomesticFixedCharge2();
    }
}
```

- **TOUConsumer.java**

```
class TOUConsumer extends Consumer {
    private double dayConsumption;
    private double peakConsumption;
    private double offPeakConsumption;

    public TOUConsumer(String id, String name, double solarExport, double
day, double peak, double off) {
        super(id, name, solarExport);
        this.dayConsumption = day;
        this.peakConsumption = peak;
        this.offPeakConsumption = off;
    }

    @Override
    public double calculateEnergyCharge(TariffRate rates) {
        return (dayConsumption * rates.getTOUDayRate()) +
            (peakConsumption * rates.getTOUPeakRate()) +
            (offPeakConsumption * rates.getTOUOffPeakRate());
    }

    @Override
    public double getFixedCharge(TariffRate rates) {
        return rates.getTOUFixedCharge();
    }
}
```

- **BillingService.java**

```
public interface BillingService {
    double calculateTotalBill(Consumer c, TariffRate r);
    void printBill(Consumer c, TariffRate r);
}
```

- **ResidentialBillingEngine.java**

```
class ResidentialBillingEngine implements BillingService {

    @Override
    public double calculateTotalBill(Consumer c, TariffRate r) {
        double energyCharge = c.calculateEnergyCharge(r);

        double fixedCharge = c.getFixedCharge(r);

        double solarCredit = c.getSolarExportUnits() *
r.getSolarExportRate();

        return (energyCharge + fixedCharge) - solarCredit;
    }

    @Override
    public void printBill(Consumer c, TariffRate r) {
        double energy = c.calculateEnergyCharge(r);
        double fixed = c.getFixedCharge(r);
        double solar = c.getSolarExportUnits() * r.getSolarExportRate();
        double net = calculateTotalBill(c, r);

        System.out.println("\n===== MONTHLY ELECTRICITY BILL =====");
        System.out.println("Consumer: " + c.getName() + " (" +
c.getConsumerID() + ")");

        if (c instanceof TOUConsumer) {
            System.out.println("Tariff Type: Time-of-Use (TOU)");
        } else {
            System.out.println("Tariff Type: Standard Domestic");
        }

        System.out.println("-----");
        System.out.printf("Energy Charge:           Rs. %10.2f%n", energy);
        System.out.printf("Fixed Charge:           Rs. %10.2f%n", fixed);
        System.out.printf("Solar Credit:          -Rs. %10.2f%n", solar);
        System.out.println("-----");
        System.out.printf("NET PAYABLE AMOUNT:    Rs. %10.2f%n", net);
        System.out.println("=====\\n");
    }
}
```

4. Implementation Details

Calculation Logic

Tariff Table for a 30 Day Billing Cycle												Annex - 2	
		Tariff effective until 14th October 2025				Approved Tariff, Effective from 15th October 2025							
		Energy Charge (LKR/kWh)		Fixed Charge (LKR/month)		Energy Charge (LKR/kWh)				Fixed Charge (LKR/month)			
Domestic													
Consumption 0-60 kWh per month													
Block 1 : 0-30 kWh		4.50		80.00		4.50				80.00			
Block 2 : 31-60 kWh		8.00		210.00		8.00				210.00			
Consumption above 60kWh per month													
Block 1 : 0-60 kWh		12.75		-		12.75				-			
Block 2 : 61-90 kWh		18.50		400.00		18.50				400.00			
Block 3 : 91-120 kWh		24.00		1,000.00		24.00				1,000.00			
Block 4 : 121-180 kWh		41.00		1,500.00		41.00				1,500.00			
Block 5 : Above 180 kWh		61.00		2,100.00		61.00				2,100.00			
Domestic Time Of Use													
Peak [18:30 to 22:30]		67.00				67.00							
Day [05:30 to 18:30]		35.00		2,100.00		35.00				2,100.00			
Off Peak [22:30 to 05:30]		21.00				21.00							
Religious & Charitable													
Block 1 : 0-30 kWh		4.50		75.00		4.50				75.00			
Block 2 : 31-90 kWh		4.50		200.00		4.50				200.00			
Block 3 : 91-120 kWh		8.00		350.00		8.00				350.00			
Block 4 : 121-180 kWh		19.00		1,300.00		19.00				1,300.00			
Block 5 : Above 180 kWh		26.00		1,700.00		26.00				1,700.00			
Other Consumers													
		Industrial/Hotel		General Purpose/Government		Industrial		Hotel		General Purpose		Government	
Volume Differentiated Monthly Consumption (kWh/month)		<300 >300		<180 >180		<300 >300		<300 >300		<180 >180		<180 >180	
Supply at 400/230V & Contract Demand <42kVA (1)		Energy Charge (LKR/kWh)		8.00 17.00		25.00 34.00		8.00 17.00		25.00 34.00		8.00 17.00	
		Fixed Charge (LKR/month)		300.00 800.00		500.00 1,600.00		300.00 800.00		500.00 1,600.00		300.00 800.00	
Supply at 400/230V & Contract Demand >42kVA (2)		Peak [18:30 to 22:30] (LKR/kWh)		28.00 47.00		28.00 47.00		28.00 47.00		28.00 47.00		28.00 47.00	
		Day [05:30 to 18:30] (LKR/kWh)		15.00 41.00		15.00 41.00		15.00 41.00		15.00 41.00		15.00 41.00	
		Off Peak [22:30 to 05:30] (LKR/kWh)		12.00 31.00		12.00 31.00		12.00 31.00		12.00 31.00		12.00 31.00	
		Demand Charge (LKR/kVA)		1,400.00 1,500.00		1,400.00 1,500.00		1,400.00 1,500.00		1,400.00 1,500.00		1,400.00 1,500.00	
		Fixed Charge (LKR/month)		5,000.00 5,000.00		5,000.00 5,000.00		5,000.00 5,000.00		5,000.00 5,000.00		5,000.00 5,000.00	
Supply at 11kV & above (3)		Peak [18:30 to 22:30] (LKR/kWh)		27.00 46.00		27.00 46.00		27.00 46.00		27.00 46.00		27.00 46.00	
		Day [05:30 to 18:30] (LKR/kWh)		14.00 39.50		14.00 39.50		14.00 39.50		14.00 39.50		14.00 39.50	
		Off Peak [22:30 to 05:30] (LKR/kWh)		11.00 30.00		11.00 30.00		11.00 30.00		11.00 30.00		11.00 30.00	
		Demand Charge (LKR/kVA)		1,350.00 1,450.00		1,350.00 1,450.00		1,350.00 1,450.00		1,350.00 1,450.00		1,350.00 1,450.00	
		Fixed Charge (LKR/month)		5,000.00 5,000.00		5,000.00 5,000.00		5,000.00 5,000.00		5,000.00 5,000.00		5,000.00 5,000.00	
Street Lighting													
Energy Charge (LKR/kWh)				50.00				50.00				50.00	
Agriculture: Optional Time of Use													
Peak [18:30 to 22:30] (LKR/kWh)				23.00				23.00				23.00	
Day [05:30 to 18:30] (LKR/kWh)				13.00				13.00				13.00	
Off Peak [22:30 to 05:30] (LKR/kWh)				7.00				7.00				7.00	
Fixed Charge (LKR/month)				750.00				750.00				750.00	
EV Charging Stations													
		DC Fast Charging		AC Level 2 Charging		DC Fast Charging				AC Level 2 Charging			
Peak (LKR/kWh)		111.00		90.00		111.00				90.00			
Day (LKR/kWh)		87.00		70.00		87.00				70.00			
Off Peak (LKR/kWh)		53.00		40.00		53.00				40.00			
Optional Pre-paid Tariff Scheme (LECO) for Retail Consumers													
		Tariff effective until 14th October 2025				Approved Tariff, Effective from 15th October 2025							
		Energy Charge (LKR/kWh)				Energy Charge (LKR/kWh)							
Domestic	Block 1 : 0-90 kWh/month					16.00							
	Block 2 : Above 90 kWh/month					61.00							
Religious	Block 1 : 0-90 kWh /month					8.00							
	Block 2 : Above 90 kWh/month					28.00							
General (GP-1)						36.00							
Industrial (I-1)						18.00							
Hotel (H-1)						18.00							



- System used latest approved tariff rates effective from 15th October 2025 from Public Utilities Commission of Sri Lanka.

- The system follows a specific mathematical formula to determine the Net Payable Amount:

$$\{\text{Net Bill}\} = (\{\text{Energy Charge}\} + \{\text{Fixed Charge}\}) - \{\text{Solar Credit}\}$$

- **Energy Charge (Standard):** Uses a multi-tiered approach. For example, if units are 60, it applies `above60Block1Rate` for the first 60 units and tiered rates for subsequent units.
- **Energy Charge (TOU):** Calculated as $(\text{Day} \times 35.00) + (\text{Peak} \times 67.00) + (\text{OffPeak} \times 21.00)$.
- **Solar Credit:** Calculated by multiplying `solarExportUnits` by a fixed rate of Rs. 25.00/kWh.

Key OOP Features Used

- **Polymorphism:** The `ResidentialBillingEngine` calls `c.calculateEnergyCharge(r)` on a `Consumer` reference. At runtime, Java determines whether to execute the **Standard** or **TOU** logic based on the actual object type.
- **Separation of Concerns:** The `TariffRate` class ensures that if electricity rates change in the future, only one file needs modification without altering the calculation logic.

5. Project Team Work

Group Number: 22

Student Number	Student Name	Contribution
EC/2023/071	K. L. S. D. Fernando	ElecBillCalculator.java
EC/2023/044	W. P. J. Prabhashwara	Consumer.java TOUConsumer.java
EC/2023/078	M. A. M. Deepal	TariffRate.java
EC/2023/014	D. D. S. V. De Alwis	StandardConsumer.java
EC/2023/037	E. K. N. S. Ekanayake	BillingService.java ResidentialBillingEngine.java

6. Conclusion

The Modern Residential Electricity Bill Calculator successfully automates the complex domestic billing process. By leveraging Object-Oriented principles, the team has created a scalable system that accurately handles different tariff plans and solar credits, providing users with a transparent breakdown of their utility expenses.